Q1. Define Natural Language Processing (NLP). Provide three real-world applications of NLP and explain how they impact society.

Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on the interaction between computers and human languages. It involves programming computers to process, analyze, and understand large amounts of natural language data.

Real-World Applications & Impact:

- **Machine Translation (e.g., Google Translate)** = Breaks down language barriers, allowing for global communication and access to information in various languages.
- **Sentiment Analysis** = Used by companies to analyze customer feedback or social media trends, helping businesses understand public perception and improve services.
- **Virtual Assistants (e.g., Siri, Alexa)** = Enhances accessibility and efficiency by allowing users to interact with technology using natural voice commands

Q2. Explain the following terms and their significance in NLP:

1. Tokenization
2. Stemming
3. Lemmatization

- **Tokenization** = The process of breaking down a text into smaller units called "tokens," such as words or sentences. It is the foundational step for almost all NLP tasks.
- **Stemming** = A rule-based process that chops off the ends of words to find the "root" form (e.g., "running" becomes "run"). It helps in normalizing words, though it can sometimes result in non-dictionary words.
- **Lemmatization** = Similar to stemming but more sophisticated; it uses vocabulary and morphological analysis to return the dictionary form of a word, known as a lemma (e.g., "better" becomes "good").

Q3. What is Part-of-Speech (POS) tagging? Discuss its importance with an example.

POS tagging is the process of marking up a word in a text as corresponding to a particular part of speech (noun, verb, adjective, etc.) based on its definition and context.
Importance: It is crucial for understanding the grammatical structure and meaning of a sentence. For example, in the sentences "I can a fruit" vs. "I have a can," POS tagging identifies the first "can" as a verb and the second as a noun, which is vital for accurate translation or summarization.

Q4. Create a TextBlob named exercise blob containing ”This is a TextBlob”.

In [2]:

```
from textblob import TextBlob
exercise_blob = TextBlob("This is a TextBlob")
exercise_blob
```

Out[2]:

```
TextBlob("This is a TextBlob")
```

Q5. Write a Python script to perform the following tasks on the given text:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Tokenize the text into words and sentences.
- Perform stemming and lemmatization using NLTK or SpaCy.
- Remove stop words from the text.
- Sample Text: "Natural Language Processing enables machines to understand and process human languages. It is a fascinating field with numerous applications, such as chatbots and language translation."

In [5]:

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Natural Language Processing enables machines to understand and proces

doc = nlp(text)

words = [token.text for token in doc]
sentences = [sent.text for sent in doc.sents]

filtered_tokens = [token for token in doc if not token.is_stop and not token.
lemmatized_words = [token.lemma_ for token in filtered_tokens]

print("Original Words:", words)
print("Sentences:", sentences)
print("Filtered & Lemmatized Words:", lemmatized_words)
```

```
Original Words: ['Natural', 'Language', 'Processing', 'enables', 'machines',
'to', 'understand', 'and', 'process', 'human', 'languages', '.', 'It', 'is',
'a', 'fascinating', 'field', 'with', 'numerous', 'applications', ',', 'such',
'as', 'chatbots', 'and', 'language', 'translation', '.']
Sentences: ['Natural Language Processing enables machines to understand and pr
ocess human languages.', 'It is a fascinating field with numerous application
s, such as chatbots and language translation.']
Filtered & Lemmatized Words: ['Natural', 'Language', 'processing', 'enable',
'machine', 'understand', 'process', 'human', 'language', 'fascinating', 'fiel
d', 'numerous', 'application', 'chatbot', 'language', 'translation']
```

Q6. Web Scraping with the Requests and Beautiful Soup Libraries:

- Use the requests library to download the www.python.org home page's content.
- Use the Beautiful Soup library to extract only the text from the page.
- Eliminate the stop words in the resulting text, then use the wordcloud module to create a wordcloud based on the text.

In [9]:

```
import requests
from bs4 import BeautifulSoup
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from nltk.corpus import stopwords

res = requests.get('https://www.python.org')
soup = BeautifulSoup(res.text, 'html.parser')
raw_text = soup.get_text()

stops = set(stopwords.words('english'))
wc = WordCloud(stopwords=stops, background_color="white").generate(raw_text)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.imshow(wc)
plt.axis("off")
plt.show()
```

Q7. (Tokenizing Text and Noun Phrases) Using the text from above problem, create a TextBlob, then tokenize it into Sentences and Words, and extract its noun phrases.

In [11]:

```
from textblob import TextBlob

blob = TextBlob(raw_text)
sentences = blob.sentences
words = blob.words
noun_phrases = blob.noun_phrases

print(sentences[:2])
print(words[:2])
print(noun_phrases[:2])
```

```
[Sentence("
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Welcome to Python.org
```

```
Notice: This page displays a fallback because interactive scripts did not ru
n."), Sentence("Possible causes include disabled JavaScript or failure to load
scripts or stylesheets.")]
['Welcome', 'to']
['welcome', 'python.org notice']
```

Q8. (Sentiment of a News Article) Using the techniques in problem no. 5, download a web page for a current news article and create a TextBlob. Display the sentiment for the entire TextBlob and for each Sentence.

In [14]:

```python
import requests
from bs4 import BeautifulSoup
from textblob import TextBlob

url = 'https://www.bbc.com/news/articles/clyp24j7nqdo'
html = requests.get(url).text
soup = BeautifulSoup(html, 'html.parser')
text = soup.get_text()

blob = TextBlob(text)
print(blob.sentiment)

for sentence in blob.sentences:
    print(sentence.sentiment)
```

```
Sentiment(polarity=0.12429527836504581, subjectivity=0.37764857881136954)
Sentiment(polarity=0.0, subjectivity=0.0)
Sentiment(polarity=0.0, subjectivity=0.0)
Sentiment(polarity=0.05576923076923078, subjectivity=0.3378205128205129)
Sentiment(polarity=0.18409090909090908, subjectivity=0.3916666666666666)
Sentiment(polarity=0.2, subjectivity=0.2)
Sentiment(polarity=0.171969696969697, subjectivity=0.4241161616161616)
Sentiment(polarity=0.0, subjectivity=0.0)
Sentiment(polarity=-0.05, subjectivity=0.325)
Sentiment(polarity=0.0, subjectivity=0.1)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Q9. (Sentiment of a News Article with the NaiveBayesAnalyzer) Repeat the previous exercise but use the NaiveBayesAnalyzer for sentiment analysis.

In [15]:

```
from textblob import TextBlob
from textblob.sentiments import NaiveBayesAnalyzer

blob_nb = TextBlob(text, analyzer=NaiveBayesAnalyzer())
print(blob_nb.sentiment)

for sentence in blob_nb.sentences:
    print(sentence.sentiment)
```

```
Sentiment(classification='pos', p_pos=1.0, p_neg=2.6187586498026565e-21)
Sentiment(classification='neg', p_pos=0.3204291660223993, p_neg=0.679570833977
6017)
Sentiment(classification='pos', p_pos=0.5, p_neg=0.5)
Sentiment(classification='pos', p_pos=0.9999999868756934, p_neg=1.312431356848
1826e-08)
Sentiment(classification='pos', p_pos=0.9994869643714028, p_neg=0.000513035628
6055419)
Sentiment(classification='pos', p_pos=0.9995227763552366, p_neg=0.000477223644
75887095)
Sentiment(classification='pos', p_pos=0.9999999999071716, p_neg=9.283159316493
604e-11)
Sentiment(classification='pos', p_pos=0.6473189248463977, p_neg=0.352681075153
60264)
Sentiment(classification='pos', p_pos=0.9103077734528857, p_neg=0.089692226547
1128)
Sentiment(classification='neg', p_pos=0.42886703710495666, p_neg=0.57113296289
50439)
```

Q10. (Spell Check a Project Gutenberg Book) Download a Project Gutenberg book and create a TextBlob. Tokenize the TextBlob into Words and determine whether any are misspelled. If so, display the possible corrections

In [16]:

```
import requests
from textblob import TextBlob
from textblob import Word

url = "https://www.gutenberg.org/files/1342/1342-0.txt"
text = requests.get(url).text[:1000]
blob = TextBlob(text)

for word in blob.words:
    w = Word(word.lower())
    suggestions = w.spellcheck()
    if suggestions[0][1] < 1.0:
        print(w, suggestions)
```

```
ruskin [('skin', 0.4150537634408602), ('russian', 0.33046594982078853), ('russ
ia', 0.13835125448028673), ('tushin', 0.053763440860215055), ('ruin', 0.034408
602150537634), ('rushing', 0.014336917562724014), ('muslin', 0.003584229390681
0036), ('rustic', 0.002150537634408602), ('russen', 0.0014336917562724014),
('risking', 0.0014336917562724014), ('resin', 0.0014336917562724014), ('austi
n', 0.0014336917562724014), ('rustan', 0.0007168458781362007), ('russie', 0.00
07168458781362007), ('pushkin', 0.0007168458781362007)]
saintsbury [('saintsbury', 0.0)]
ruskin [('skin', 0.4150537634408602), ('russian', 0.33046594982078853), ('russ
ia', 0.13835125448028673), ('tushin', 0.053763440860215055), ('ruin', 0.034408
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
602150537634), ('rushing', 0.014336917562724014), ('muslin', 0.003584229390681
0036), ('rustic', 0.002150537634408602), ('russen', 0.0014336917562724014),
('risking', 0.0014336917562724014), ('resin', 0.0014336917562724014), ('austi
n', 0.0014336917562724014), ('rustan', 0.0007168458781362007), ('russie', 0.00
07168458781362007), ('pushkin', 0.0007168458781362007)]
```

Q11.

- Write a Python program that takes user input in English and translates it to French, Spanish, and German using TextBlob.
- Create a program that takes multiple user-inputted sentences, analyzes polarity and subjectivity, and categorizes them as objective/subjective and positive/negative/neutral Develop a function that takes a paragraph, splits it into sentences, and calculates the sentiment score for each sentence individually.
- Write a program that takes a sentence as input and prints each word along with its POS tag using TextBlob.
- Create a function that takes a user-inputted word, checks its spelling using TextBlob, and suggests top 3 closest words if a mistake is found.
- Build a Python script that extracts all adjectives from a given paragraph and prints them in order of occurrence.
- Write a program that takes a news article as input and extracts the top 5 most common noun phrases as keywords.
- Write a program that takes a news article as input and extracts the top 5 most common noun phrases as keywords.
- Write a program that summarizes a given paragraph by keeping only the most informative sentences, based on noun phrase frequency.

In [20]:

```python
from textblob import TextBlob, Word
from collections import Counter
import nltk
from deep_translator import GoogleTranslator

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

def translate_text():
    text = input("Enter English text: ")

    print("French:", GoogleTranslator(source='en', target='fr').translate(tex
    print("Spanish:", GoogleTranslator(source='en', target='es').translate(te
    print("German:", GoogleTranslator(source='en', target='de').translate(tex

def sentiment_analysis():
    n = int(input("How many sentences? "))
    for i in range(n):
        s = input(f"Sentence {i+1}: ")
        blob = TextBlob(s)

        polarity = blob.sentiment.polarity
        subjectivity = blob.sentiment.subjectivity

        if polarity > 0:
            sentiment = "Positive"
        elif polarity < 0:
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            sentiment = "Negative"
        else:
            sentiment = "Neutral"

        obj_type = "Subjective" if subjectivity > 0.5 else "Objective"

        print(f"\n{s}")
        print("Sentiment:", sentiment)
        print("Type:", obj_type)

def sentence_sentiment():
    text = input("Enter paragraph: ")
    blob = TextBlob(text)

    for sentence in blob.sentences:
        print(sentence)
        print("Sentiment:", sentence.sentiment)

def pos_tagging():
    text = input("Enter sentence: ")
    blob = TextBlob(text)

    for word, tag in blob.tags:
        print(word, "->", tag)

def spell_check():
    word = input("Enter word: ")
    w = Word(word)

    if w.spellcheck()[0][0] == word:
        print("Correct spelling")
    else:
        print("Top suggestions:")
        for s in w.spellcheck()[:3]:
            print(s[0])

def extract_adjectives():
    text = input("Enter paragraph: ")
    blob = TextBlob(text)

    adjectives = [w for w, t in blob.tags if t.startswith("JJ")]
    print("Adjectives:", adjectives)

def noun_phrases():
    text = input("Enter article: ")
    blob = TextBlob(text)

    counts = Counter(blob.noun_phrases)

    print("Top 5 noun phrases:")
    for w, c in counts.most_common(5):
        print(w, ":", c)

def summarize():
    text = input("Enter paragraph: ")
    blob = TextBlob(text)

    np_counts = Counter(blob.noun_phrases)
    scores = {}

    for sentence in blob.sentences:
        score = 0
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        for np in np_counts:
            if np in sentence.lower():
                score += np_counts[np]
        scores[sentence] = score

    top = sorted(scores, key=scores.get, reverse=True)[:3]

    print("\nSummary:")
    for s in top:
        print(s)

while True:
    print("\n===== NLP MENU =====")
    print("1. Translate Text")
    print("2. Sentiment Analysis (multiple sentences)")
    print("3. Sentence-wise Sentiment")
    print("4. POS Tagging")
    print("5. Spell Check")
    print("6. Extract Adjectives")
    print("7. Noun Phrase Keywords")
    print("8. Summarization")
    print("9. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        translate_text()
    elif choice == "2":
        sentiment_analysis()
    elif choice == "3":
        sentence_sentiment()
    elif choice == "4":
        pos_tagging()
    elif choice == "5":
        spell_check()
    elif choice == "6":
        extract_adjectives()
    elif choice == "7":
        noun_phrases()
    elif choice == "8":
        summarize()
    elif choice == "9":
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

```
===== NLP MENU =====
1. Translate Text
2. Sentiment Analysis (multiple sentences)
3. Sentence-wise Sentiment
4. POS Tagging
5. Spell Check
6. Extract Adjectives
7. Noun Phrase Keywords
8. Summarization
9. Exit
```

```
[nltk_data] Downloading package punkt to /home/iteradmin/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /home/iteradmin/nltk_data...
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
Enter choice: 1
Enter English text: hello i am human
French: bonjour, je suis humain
Spanish: hola soy humano
German: Hallo, ich bin ein Mensch

===== NLP MENU =====
1. Translate Text
2. Sentiment Analysis (multiple sentences)
3. Sentence-wise Sentiment
4. POS Tagging
5. Spell Check
6. Extract Adjectives
7. Noun Phrase Keywords
8. Summarization
9. Exit
Enter choice: 2
How many sentences? 1
Sentence 1: hello i am human

hello i am human
Sentiment: Neutral
Type: Objective

===== NLP MENU =====
1. Translate Text
2. Sentiment Analysis (multiple sentences)
3. Sentence-wise Sentiment
4. POS Tagging
5. Spell Check
6. Extract Adjectives
7. Noun Phrase Keywords
8. Summarization
9. Exit
Enter choice: 9
Exiting...
```

Q12. Write a Python program that takes a word as input and returns:

- Its definition
- Its synonyms
- Its antonyms(if available)

In [22]:

```python
from nltk.corpus import wordnet

def word_info(word):
    synonyms = set()
    antonyms = set()
    definition = None

    for syn in wordnet.synsets(word):
        if definition is None:
            definition = syn.definition()

        for lemma in syn.lemmas():
            synonyms.add(lemma.name())

            if lemma.antonyms():
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
                for ant in lemma.antonyms():
                    antonyms.add(ant.name())

    print("\nWord:", word)

    print("\nDefinition:")
    print(definition if definition else "No definition found")

    print("\nSynonyms:")
    print(", ".join(synonyms) if synonyms else "None found")

    print("\nAntonyms:")
    print(", ".join(antonyms) if antonyms else "None found")



w = input("Enter a word: ")
word_info(w)
```

```
Enter a word: good

Word: good

Definition:
benefit

Synonyms:
in_effect, just, dependable, soundly, practiced, adept, proficient, safe, wel
l, serious, estimable, goodness, right, near, thoroughly, honest, secure, skil
lful, unspoiled, good, in_force, commodity, full, dear, unspoilt, salutary, ho
norable, ripe, upright, beneficial, effective, undecomposed, trade_good, exper
t, respectable, skilful, sound

Antonyms:
evil, badness, evilness, ill, bad
```

Q13.

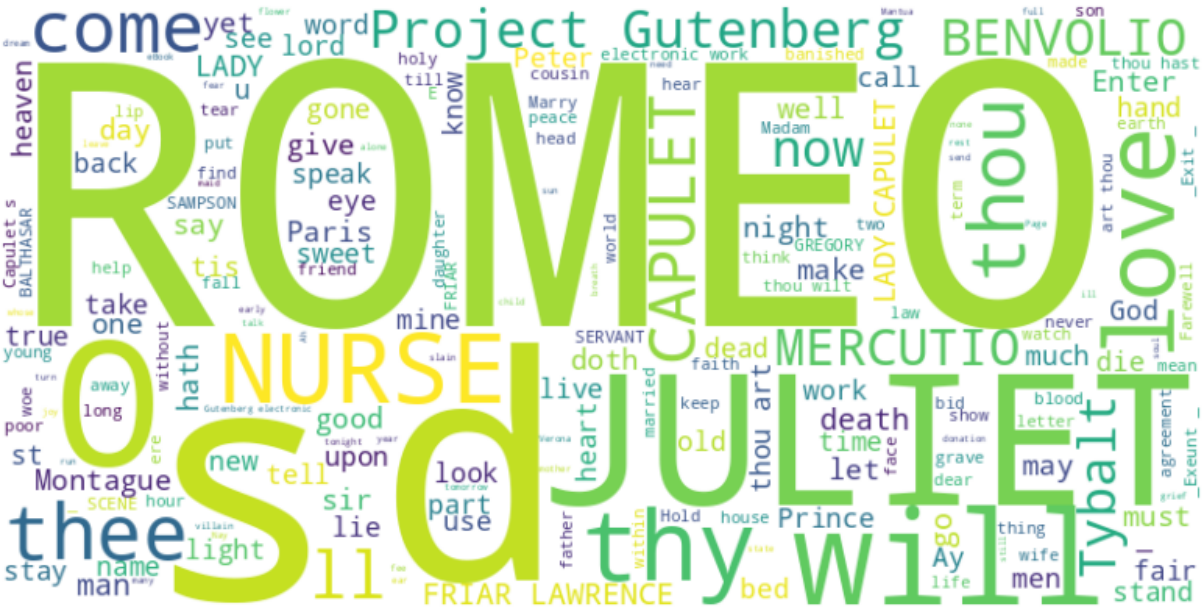
- Write a Python program that reads a .txt file, processes the text, and generates a word cloud visualization.
- Create a word cloud in the shape of an object (e.g., a heart, star) using WordCloud and a mask image.

In [30]:

```
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

file_path = "RomeoAndJuliet.txt"

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

stopwords = set(STOPWORDS)

wc = WordCloud(
    width=800,
    height=400,
    background_color="white",
    stopwords=stopwords
).generate(text)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.figure(figsize=(10,5))
plt.imshow(wc, interpolation="bilinear")
plt.axis("off")
plt.show()
```


ROMEO
JULIET
NURSE
Project Gutenberg
BENVOLIO
MERCUTIO
CAPULET
FRIAR LAWRENCE
come
thou
love
thy
will
thee
Tybalt
Montague
Paris
Enter
now
night
death
heaven
day
back
give
speak
eye
sweet
say
take
one
true
mine
live
doth
dead
work
time
let
good
upon
look
part
use
sir
lie
light
name
man
stay
new
tell
hath
without
away
long
poor
woe
st
Capulet
BALTHASAR
SAMPSON
LADY
CAPULET
GREGORY
SERVANT
Peter
Marry
peace
head
cousin
electronic work
banished
call
well
Madam
hand
earth
thou hast
made
son
Farewell
Exit
Exeunt
agreement
mean
die
much
never
watch
God
must
may
fair
stand
men
Ay
wife
thing
Prince
house
Hold
bed
father
within
married
keep
old
thou art
heart
blood
letter
show
bid
grave
dear
face
donation
life
art thou
term
world
two
think
make
thou wilt
law
child
breath
Page
rest
sand
none
Mantua
full
dream
ebook
flower
yet
see
lord
word
holy
till
know
daughter
FRIAR
gone
put
find
tear
lip
alone
tis
fall
help
friend
early
talk
Gutenberg electronic
tonight
year
verona
mother
villain
hour
SCENE
hear
u
LADY
fear
dear
fair
fan
ear
slain
child
sun
breath
Page
term
rest
sand
donation
grief
Exeunt
Exit
art thou
agreement
Farewell
mean
thou hast
made
son
Mantua
full
banished
work
electronic
cousin
Peter
holy
till
know
Marry
peace
head
Madam
well
call
hand
earth
Enter
thou
love
God
never
watch
much
die
must
may
fair
stand
men
Ay
wife
thing
life
donation
face
let
time
death
grave
dear
show
bid
letter
blood
keep
old
thou art
heart
live
doth
dead
work
SERVANT
GREGORY
make
night
two
think
thou wilt
law
child
breath
world
daughter
FRIAR
mine
take
one
true
young
away
long
poor
woe
st
Montague
SCENE
hour
villain
stay
name
man
light
new
tell
upon
good
hath
without
ere
joy
turn
run
Gutenberg electronic
tonight
year
verona
mother
slain
SAMPSON
LADY
CAPULET
Capulet
BALTHASAR
say
help
tis
fall
friend
eye
sweet
Paris
speak
give
gone
put
find
tear
lip
alone
day
back
heaven
u
LADY
see
lord
word
yet
come
dream
ebook
flower

Q14. (Textatistic: Readability of News Articles) Using the above techniques, download from several news sites current news articles on the same topic. Perform readability assessments on them to determine which sites are the most readable. For each article, calculate the average number of words per sentence, the average number of characters per word and the average number of syllables per word.

In [36]:

```
import re

def count_syllables(word):
    word = word.lower()
    word = re.sub(r'[^a-z]', '', word)

    vowels = "aeiouy"
    count = 0
    prev_vowel = False

    for char in word:
        is_vowel = char in vowels
        if is_vowel and not prev_vowel:
            count += 1
        prev_vowel = is_vowel
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    if word.endswith("e") and count > 1:
        count -= 1

    return max(1, count)

from nltk.tokenize import sent_tokenize, word_tokenize
import requests
from bs4 import BeautifulSoup

def analyze_article(url):
    headers = {"User-Agent": "Mozilla/5.0"}
    html = requests.get(url, headers=headers).text
    soup = BeautifulSoup(html, "html.parser")

    # extract text
    text = soup.get_text(separator=" ")

    sentences = sent_tokenize(text)
    words = word_tokenize(text)

    words = [w for w in words if w.isalpha()]

    if len(sentences) == 0 or len(words) == 0:
        return None

    # metrics
    avg_words_per_sentence = len(words) / len(sentences)

    avg_chars_per_word = sum(len(w) for w in words) / len(words)

    avg_syllables_per_word = sum(count_syllables(w) for w in words) / len(wor

    return {
        "sentences": len(sentences),
        "words": len(words),
        "avg_words_per_sentence": avg_words_per_sentence,
        "avg_chars_per_word": avg_chars_per_word,
        "avg_syllables_per_word": avg_syllables_per_word
    }

urls = [
    "https://www.bbc.com/news/world-us-canada-66231862",
    "https://www.theguardian.com/world/2024/example",
    "https://www.reuters.com/world/example"
]

for url in urls:
    print("\n====================")
    print("URL:", url)

    result = analyze_article(url)

    if result:
        print("Sentences:", result["sentences"])
        print("Words:", result["words"])
        print("Avg words/sentence:", round(result["avg_words_per_sentence"],
        print("Avg chars/word:", round(result["avg_chars_per_word"], 2))
        print("Avg syllables/word:", round(result["avg_syllables_per_word"],
    else:
        print("Could not extract article text")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
====================
URL: https://www.bbc.com/news/world-us-canada-66231862
Sentences: 17
Words: 658
Avg words/sentence: 38.71
Avg chars/word: 5.17
Avg syllables/word: 1.71

====================
URL: https://www.theguardian.com/world/2024/example
Sentences: 4
Words: 44
Avg words/sentence: 11.0
Avg chars/word: 4.07
Avg syllables/word: 1.36

====================
URL: https://www.reuters.com/world/example
Sentences: 1
Words: 8
Avg words/sentence: 8.0
Avg chars/word: 4.5
Avg syllables/word: 1.5
```

Q15. (spaCy: Named Entity Recognition) Using the above techniques, download a current news article, then use the spaCy library's named entity recognition capabilities to display the named entities (people, places, organizations, etc.) in the article.

In [38]:

```python
import requests
from bs4 import BeautifulSoup
import spacy

nlp = spacy.load("en_core_web_sm")

url = "https://www.bbc.com/news/world-us-canada-66231862"

headers = {"User-Agent": "Mozilla/5.0"}
html = requests.get(url, headers=headers).text

soup = BeautifulSoup(html, "html.parser")

paragraphs = soup.find_all("p")
text = " ".join([p.get_text() for p in paragraphs])

doc = nlp(text)

print("\n===== NAMED ENTITIES =====\n")

for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
===== NAMED ENTITIES =====

Jayne Charles's -> PERSON
third -> ORDINAL
the Wolverhampton MS Therapy Centre -> ORG
Jay House -> PERSON
two weeks -> DATE
Aberystwyth -> ORG
Hereford -> PERSON
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Veolia -> GPE
Fenn's Bank -> ORG
Wrexham -> PERSON
A year -> DATE
Beryl -> GPE
Bertie -> PERSON
Shrewsbury -> WORK_OF_ART
Old River Bed -> LOC
2026 -> DATE
BBC -> ORG
BBC -> ORG
```

Q16. (spaCy: Shakespeare Similarity Detection) Using the spaCy techniques, download a Shakespeare comedy from Project Gutenberg and compare it for similarity with Romeo and Juliet.

In [40]:

```
import requests
import spacy
from bs4 import BeautifulSoup

nlp = spacy.load("en_core_web_md")

def get_text(url):
    headers = {"User-Agent": "Mozilla/5.0"}
    html = requests.get(url, headers=headers).text
    soup = BeautifulSoup(html, "html.parser")
    return soup.get_text()

def clean(text):
    return " ".join(text.split())

romeo_url = "https://www.gutenberg.org/cache/epub/1513/pg1513.txt"
romeo_text = clean(get_text(romeo_url))

comedy_url = "https://www.gutenberg.org/cache/epub/1514/pg1514.txt"
comedy_text = clean(get_text(comedy_url))

doc1 = nlp(romeo_text[:200000])
doc2 = nlp(comedy_text[:200000])

similarity_score = doc1.similarity(doc2)

print("\n===== SHAKESPEARE SIMILARITY =====")
print("Romeo & Juliet vs Comedy")
print("Similarity Score:", similarity_score)
```

```
===== SHAKESPEARE SIMILARITY =====
Romeo & Juliet vs Comedy
Similarity Score: 0.9980871677398682
```

Q17. (textblob.utils Utility Functions) Use strip punc and lowerstrip functions of TextBlob's textblob.utils module with all=True keyword argument to remove punctuation and to get a string in all lowercase letters with whitespace and punctuation removed. Experiment with each function on Romeo and Juliet.

In [41]:

```
from pathlib import Path
from textblob import TextBlob
from textblob.utils import strip_punc, lowerstrip

text = Path("RomeoAndJuliet.txt").read_text(encoding="utf-8")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
blob = TextBlob(text)

print("\n===== ORIGINAL TEXT SAMPLE =====\n")
print(text[:800])

clean_punc = strip_punc(text, all=True)

print("\n===== AFTER strip_punc(all=True) =====\n")
print(clean_punc[:800])

clean_lower = lowerstrip(text, all=True)

print("\n===== AFTER lowerstrip(all=True) =====\n")
print(clean_lower[:800])

print("\n===== TEXT STATISTICS =====\n")

print("Original length:", len(text))
print("After strip_punc:", len(clean_punc))
print("After lowerstrip:", len(clean_lower))

print("\n===== SAMPLE WORD COMPARISON =====\n")

print("Original words:", text.split()[:20])
print("Clean words:", clean_lower.split()[:20])
```

```
===== ORIGINAL TEXT SAMPLE =====

The Project Gutenberg eBook of Romeo and Juliet

This eBook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever. You may copy it, give it away or re-use it under the terms
of the Project Gutenberg License included with this eBook or online
at www.gutenberg.org. If you are not located in the United States,
you will have to check the laws of the country where you are located
before using this eBook.

Title: Romeo and Juliet

Author: William Shakespeare



Release date: November 1, 1998 [eBook #1513]
                Most recently updated: September 18, 2025

Language: English

Other information and formats: www.gutenberg.org/ebooks/1513

Credits: the PG Shakespeare Team, a team of about twen

===== AFTER strip_punc(all=True) =====

The Project Gutenberg eBook of Romeo and Juliet

This eBook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever You may copy it give it away or reuse it under the terms
of the Project Gutenberg License included with this eBook or online
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

at wwwgutenbergorg If you are not located in the United States
you will have to check the laws of the country where you are located
before using this eBook

Title Romeo and Juliet

Author William Shakespeare

Release date November 1 1998 eBook 1513
Most recently updated September 18 2025

Language English

Other information and formats wwwgutenbergorgebooks1513

Credits the PG Shakespeare Team a team of about twenty Project Gutenberg volu

===== AFTER lowerstrip(all=True) =====

the project gutenberg ebook of romeo and juliet

this ebook is for the use of anyone anywhere in the united states and
most other parts of the world at no cost and with almost no restrictions
whatsoever you may copy it give it away or reuse it under the terms
of the project gutenberg license included with this ebook or online
at wwwgutenbergorg if you are not located in the united states
you will have to check the laws of the country where you are located
before using this ebook

title romeo and juliet

author william shakespeare

release date november 1 1998 ebook 1513
most recently updated september 18 2025

language english

other information and formats wwwgutenbergorgebooks1513

credits the pg shakespeare team a team of about twenty project gutenberg volu

===== TEXT STATISTICS =====

Original length: 161818
After strip_punc: 154422
After lowerstrip: 154422

===== SAMPLE WORD COMPARISON =====

Original words: ['The', 'Project', 'Gutenberg', 'eBook', 'of', 'Romeo', 'and',
'Juliet', 'This', 'eBook', 'is', 'for', 'the', 'use', 'of', 'anyone', 'anywher
e', 'in', 'the', 'United']
Clean words: ['the', 'project', 'gutenberg', 'ebook', 'of', 'romeo', 'and', 'j
uliet', 'this', 'ebook', 'is', 'for', 'the', 'use', 'of', 'anyone', 'anywher
e', 'in', 'the', 'united']

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF